AI Engineering and Claude Architect · Part III, Agents

Agents and MCP, Lesson 3

Designing Tools an Agent Can Use

The agent had `analyze_content` and `analyze_document`, both described as "analyzes and returns the results". It picked between them more or less at random. Nobody had changed the model, the prompt or the data. The interface was the bug.

Module	AI06 of 13
Lesson	3 of 10
Prerequisites	AI06 L2, AI03 L2–L3
Exam relevance	**Direct.** All of TS 2.1 and TS 2.3
Reading time	~45 minutes

Contents

1. Designing Tools an Agent Can Use

Lesson 3 of 10 · Agents and MCP

Designing Tools an Agent Can Use

Key idea

A tool's description is a prompt, and the model chooses tools mainly by reading it. So overlap and vagueness become wrong tool calls. **Design for an agent, not for a developer: few tools with distinct purposes, the right scope per role, and results that spend context carefully.** Then evaluate, because how a tool reads is an empirical question.

> *Before: the loop (AI06 L2), prompting (AI03 L2). Teaches: writing descriptions, splitting and renaming tools, scoping per agent,* `tool_choice` *old and new, token-efficient results, and tool search. Used by: AI06 L4–L5, L9–L10, AI08 L3, AI13.*

The hook: when the interface is the bug

Anthropic's Building Effective Agents reports spending more time optimizing tools than the overall prompt on its SWE-bench agent. The model made mistakes with relative file paths after leaving the root directory. So they changed the tool to require absolute paths, and the post reports that the model then used this method flawlessly. The interface had made the error harder to make.

They call this **poka-yoke**, a term from manufacturing: "Change the arguments so that it is harder to make mistakes."

A linter can catch the mechanical problems before any model sees them. Real output from `code/ai06-agents/.../ToolLint.java`:

```
Before:
  analyze_content: description under 12 words
  analyze_content: doesn't say when to use it
  analyze_content: ambiguous parameter 'data'
  analyze_document: description under 12 words
  analyze_document: doesn't say when to use it
  analyze_document: ambiguous parameter 'id'
  analyze_content and analyze_document: 60% description overlap
After:
  no problems found
```

The linter can't say the model will choose well. "No problems found" means only that the obvious mechanical faults are gone. Only an evaluation can show selection quality.

Intuition: a new hire and a toolbox

Imagine handing a new hire two drawers labeled "Stuff" and "Things". They'd guess. Now imagine one labeled "Customer refunds under $500: use this after verifying identity. For anything larger, use Escalations."

Anthropic's Writing Tools for Agents uses the same image: write descriptions as you would for a new hire, making implicit context, niche terms and relationships explicit.

Mechanism: how a model chooses

The model sees tool names, descriptions and schemas as part of its prompt. Nothing else tells it which tool does what.

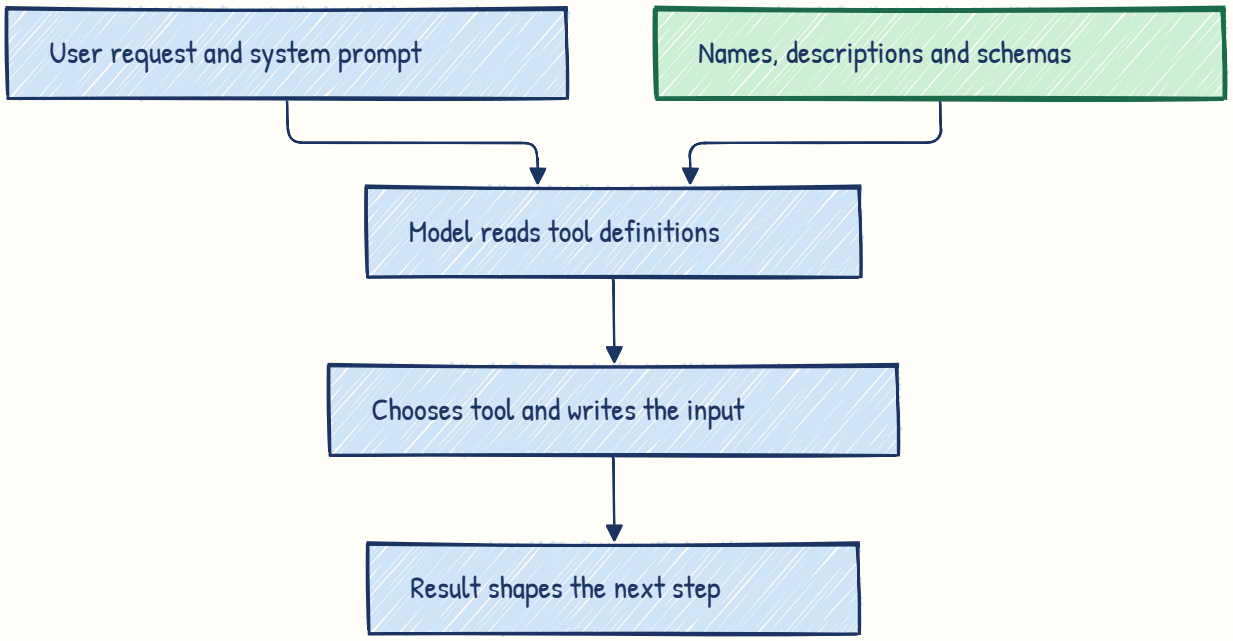


Figure 1. How a tool call gets chosen. Everything the model knows about a tool arrives through its definition, so definitions must carry the distinctions.

What a good description contains

Part	Bad	Good
What it does	"Gets customer"	"Finds one customer by exact email or phone"
What it returns	(nothing)	"Returns ID, name, verification state and open orders"
When to use it	(nothing)	"Use this first, whenever a customer gives contact details"
When not to	(nothing)	"For an order number, use `lookup_order` instead"
Input formats	`id: string`	`order_id: digits only, such as 7781`
Edge cases	(nothing)	"Returns an empty list if there's no match. That isn't an error"

Here's a definition in that style, from `ClaudeAgentLoop.java`, with `strict: true` keeping arguments schema-valid. It compiles, but hasn't been run:

```
return Tool.builder()
        .name("lookup_order")
        .description("""
                Look up one order by its ID, such as "7781". Returns status, total \
                and delivery date. Use this for questions about a specific order, \
                not for finding a customer's orders.""")
```

The schema adds `additionalProperties: false`, and the builder ends with `.strict(true)`.

Recent Opus models call tools more conservatively, and the Claude API guidance says that trigger conditions in a description ("call this when…") measurably raise the should-call rate.

Fixing overlap: rename, then split

Problem	Fix	Example
Two tools read the same	**Rename and differentiate**	`analyze_content` becomes `extract_web_results`, described as web-specific
One tool does everything	**Split by purpose**	`analyze_document` becomes `extract_data_points`, `summarize_content`, `verify_claim_against_source`
A generic tool invites misuse	**Constrain it**	`fetch_url` becomes `load_document`, which validates document URLs
Several tools chained every time	**Consolidate**	`list_users`, `list_events`, `create_event` become `schedule_event`

Consolidating and splitting aren't opposites. Split when one tool has several unrelated jobs, and consolidate when one job takes several tools.

Check your system prompt too. TS 2.1 warns that keyword-sensitive instructions can create unintended associations. "Always analyze customer messages first" may push the model towards `analyze_content`, however good its description.

How many tools per agent

The guide says too many tools (18 rather than 4–5) degrade selection, and that agents misuse tools outside their role.

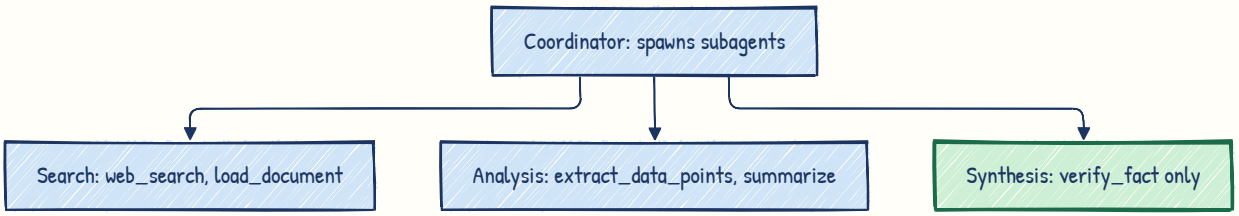


Figure 2. Scoped tools. Each role gets only what its job needs, plus one narrow cross-role tool for a frequent need. Anything complex goes back through the coordinator.

Principle	Example
Only what the role needs	Synthesis has no web search, or it will start researching
Scoped cross-role tools	A narrow `verify_fact` for a frequent need, with hard cases sent to the coordinator
Constrained over general	`load_document`, not `fetch_url`

"4–5" is the guide's illustration, not a law. Anthropic's own posts discuss far larger tool sets with tool search, below. **The principle is scope, not the number.**

`tool_choice`: the exam and today

Setting	Meaning	On Opus 5.5, Sonnet 5.5, Fable 5.1
`auto`	May call a tool, or answer in text	Works. The default
`none`	Can't call tools	Works
`any`	Must call some tool	**Returns a 400**
`{"type":"tool","name":"x"}`	Must call that tool	**Returns a 400**

Know both answers

The exam's answer: use `any` to guarantee a tool call, and forced selection to ensure one runs first, such as `extract_metadata` before enrichment. **What to build today on those models:** `auto` with an instruction naming the tool, `strict: true` for valid arguments, and a check that the call happened, retrying if it didn't. Or use `output_config.format` if you only wanted JSON. Forced selection still works on models such as Opus 5 and Fable 5.

A guaranteed sequence doesn't need forcing at all. If `extract_metadata` must run first, run it from your code before the agent starts, or use a gate (AI06 L8).

Token-efficient results

Technique	Detail	Source
Search, don't list	`search_contacts`, not `list_contacts`	Writing Tools
Return meaning	Names rather than UUIDs. Anthropic reports that this improved precision, by reducing hallucinations	Writing Tools
Offer verbosity	A `response_format` of `CONCISE` or `DETAILED`. Their example: 72 tokens vs 206	Writing Tools
Page and filter	With sensible defaults	Writing Tools
Truncate helpfully	Say how to narrow down. Claude Code limits tool responses to 25,000 tokens by default	Writing Tools
Actionable errors	"`from_date` must be YYYY-MM-DD", not a stack trace	AI06 L5

Names matter too: `user_id` is clearer than `user`. And namespacing helps across servers: `asana_search`, `jira_search`.

Why the obvious fixes fall short

Tempting move	Why it's not the first step	Better first step
Add a routing classifier	Over-engineered, and it doesn't fix the cause	Fix descriptions
Add few-shot examples of tool choice	Adds tokens around a vague interface	Fix descriptions, then test
Consolidate everything into one tool	Removes the choice, and overloads the schema	Keep distinct purposes
Wrap each API endpoint as a tool	APIs are built for programs with cheap memory	Design around tasks
Give every agent every tool	Selection degrades, and roles blur	Scope by role

How Anthropic solved it

A process, not just rules. Writing Tools describes: prototype, evaluate with realistic tasks, then let Claude analyze the transcripts and refactor the tools, testing on held-out sets. It reports that Claude-optimized tools beat expert-written ones on those sets. In the multi-agent post, a tool-testing agent that rewrote flawed descriptions cut later task completion time by 40%.

Scale. The Advanced Tool Use post, from November 2025, gave these figures:

Problem	Feature	Published result
Definitions fill context: five servers took about 55K tokens, and Anthropic saw 134K	**Tool search** with `defer_loading`	About 85% fewer tokens. Selection accuracy on their MCP evals: Opus 4 from 49% to 74%, Opus 4.5 from 79.5% to 88.1%
Intermediate results flood context	**Programmatic tool calling**	Complex research tasks: 43,588 to 27,297 tokens (37%)
Schemas don't show usage	**Tool use examples**	Complex parameter handling: 72% to 90%

These are Anthropic's own tests on older models. **Choose by your bottleneck:** context bloat, large intermediate results, or parameter errors. Its guidance is that tool search helps with 10 or more tools, or definitions over about 10K tokens, and is less useful with fewer tools.

A related post, Code Execution with MCP, describes agents calling MCP servers through code, with one example dropping from 150,000 to 2,000 tokens. Its caveat is that running generated code needs real sandboxing.

The production architecture

This is a recommended reference design, not a specific company's system.

Baseline: wrap all the APIs. It's fast to build, and context fills up with definitions.

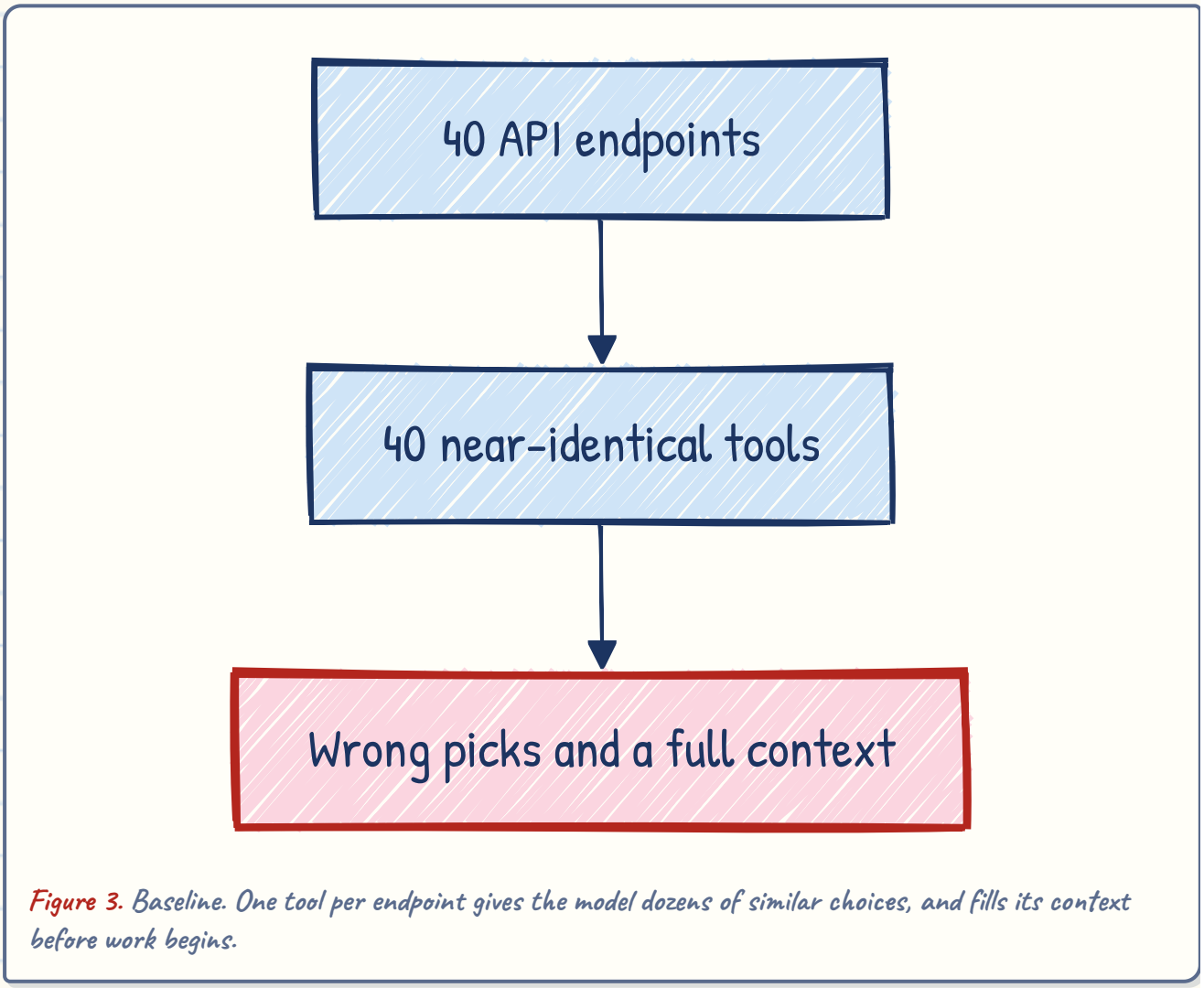


Figure 3. Baseline. One tool per endpoint gives the model dozens of similar choices, and fills its context before work begins.

Improved: task-shaped tools, scoped per role. Selection improves. New problem: **descriptions drift as tools change, and nobody notices selection getting worse.**

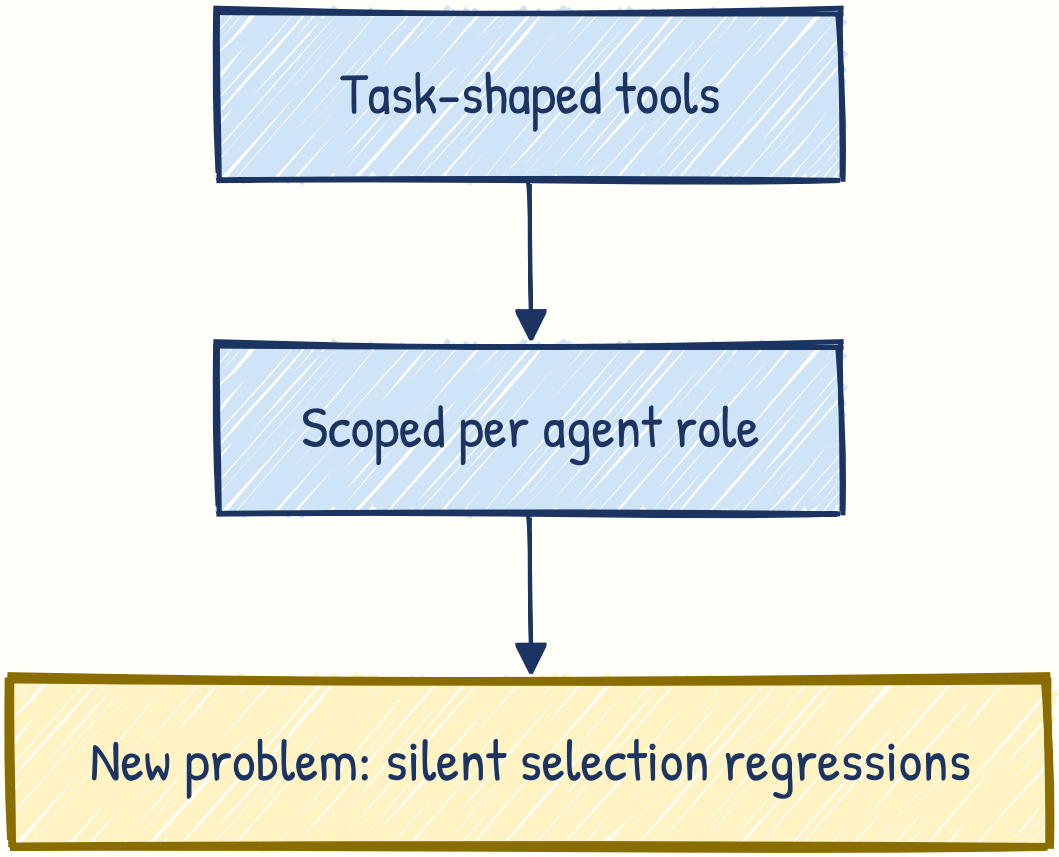


Figure 4. Improved. Fewer, clearer tools help, but nothing tells you when a change makes selection worse.

Production: tools as versioned, tested interfaces.

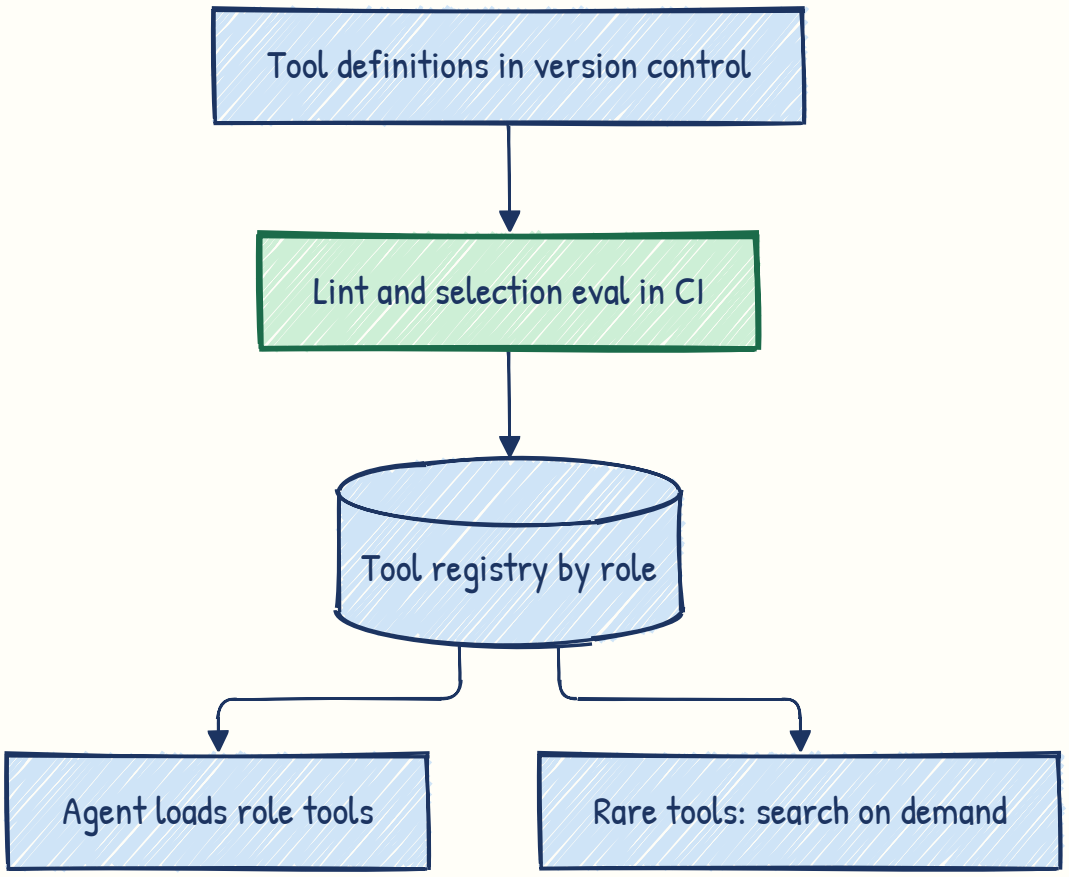


Figure 5. Production. Definitions live in a registry, pass a lint and a selection eval before release, and load per role, with rarely used tools deferred.

Component	Why it exists	How it communicates
Lint	Catches mechanical faults cheaply	CI step
Selection eval	The only true test of a description	Golden requests and expected tools
Registry	One source per role, in a stable order	Loaded at agent start. Stable order protects the prompt cache
Deferred tools	Large catalogs without the context cost	Tool search

Failures

Failure	Containment
Wrong tool chosen	Log selections, and add the case to the eval
Hallucinated parameter	`strict: true`, plus validation
Tool description changed, and selection dropped	Eval gate, and version definitions
Result too big	Truncate with guidance, and page
Tool list reordered between requests	Keep it deterministic, or the cache breaks
A tool's text injects instructions	Treat descriptions and results as untrusted (AI09 L2)

Trade-offs

Decision	Gain	Cost
Fewer tools	Better choices	Each may do more
Longer descriptions	Better selection	More tokens per request, though caching helps
Strict schemas	Valid arguments	Schema limits
Tool search	Large catalogs	A search step adds latency
Dedicated tools over bash	Can gate, audit and parallelize	Less flexibility

On that last row: a bash tool can do nearly anything, but your harness only sees an opaque string. Promote an action to a dedicated tool when you need to gate, render, audit or parallelize it.

Common mistakes

1. **One-line descriptions.** They're the main selection signal.
2. **Reaching for a router first.** Fix descriptions first.
3. **Wrapping every endpoint.** Design around tasks.
4. **Giving synthesis a search tool.** It will wander.
5. **Using forced `tool_choice` on Opus 5.5.** It returns a 400.
6. **Returning UUIDs and raw dumps.** Spend context on meaning.
7. **Trusting a linter.** Only an eval measures selection.
8. **Ignoring the system prompt.** Keywords can override descriptions.

What to say in an interview

"Descriptions are prompts, so I'd state what a tool does, returns, when to use it, and when not to. With misrouting, I'd fix descriptions before adding a classifier, then rename or split overlapping tools. I'd scope tools by role, and test selection with an eval in CI. For a guaranteed first call, the exam says forced `tool_choice` . On current Opus I'd call that step from code."

Certification lens

How this lesson maps to CCAR-F

TS 2.1 and **TS 2.3.** The proportionate first step for misrouting is better descriptions, not new infrastructure. Remember `auto` , `any` and forced selection as the guide gives them, and that current models differ.

These are original practice questions, not real or leaked exam questions.

Q Q1 · Exam style, scenario: Multi-Agent Research System · Medium. Your agent often calls `get_report` when it should call `get_report_summary`. Their descriptions are one line each and nearly identical. What's the most effective first step?

A. Add a classifier that picks the tool before each model call.

B. Add 20 few-shot examples of correct choices.

C. Merge them into one tool with many optional parameters.

D. Rewrite both descriptions to state inputs, outputs, when to use each, and how they differ.

A D (TS 2.1). It fixes the root cause, proportionately. A is over-engineering. B adds tokens around the same ambiguity. C may be valid later, but it doesn't address the cause first.

Q Q2 · Exam style, scenario: Multi-Agent Research System · Medium. The synthesis agent keeps running web searches instead of combining findings, adding latency. It has all 14 tools. What's the best change?

A. Restrict it to synthesis tools, plus a narrow `verify_fact` for quick checks, routing harder cases through the coordinator.

B. Add "don't search the web" to its prompt, and keep the tools.

C. Move it to a more capable model.

D. Run search and synthesis in the same agent.

A A (TS 2.3). Scope tools by role. B is probabilistic, and leaves the temptation in place. C doesn't address scope. D removes the separation.

Q Q3 · Exam style, scenario: Structured Data Extraction · Medium. A pipeline must guarantee that `extract_metadata` is called on the first step, with other tools following. Per the exam guide, which setting achieves this?

A. `tool_choice: "auto"` with a strongly worded prompt.

B. Forced selection of `extract_metadata` on the first request, then normal handling in follow-up turns.

C. `tool_choice: "none"` on the first request.

D. Listing `extract_metadata` first in the tools array.

A B (TS 2.3). A doesn't guarantee it. C prevents any call. D has no such effect. **Today, on Opus 5.5 and similar models,** forced selection returns a 400, so you'd call that step from code, or use `auto` with verification.

Q Q4 · Exam style, scenario: Multi-Agent Research System · Hard. A generic `analyze_document` tool gets used for extraction, summarizing and fact-checking, with inconsistent output. What should you do?

A. Add a `mode` enum with 12 values.

B. Increase `max_tokens` for this tool.

C. Split it into purpose-specific tools with clear input and output contracts.

D. Run it twice, and compare the outputs.

A **C** (TS 2.1). The guide gives this exact split as a skill. A keeps one tool doing unrelated jobs. B and D don't touch the interface.

Q Q5 · Design · Hard. Your agent has 60 tools across five MCP servers, and definitions use 50K tokens. Tool selection is often wrong. Parameters are usually right, and results are small. Which feature from Anthropic's Advanced Tool Use post fits, and why not the other two?

A **Tool search** with `defer_loading`, since the bottleneck is definitions and selection. Programmatic tool calling targets large intermediate results, which you don't have. Tool use examples target parameter errors, which you also don't have. Keep the few most-used tools always loaded.

Revision - Lesson 3

→ Descriptions are prompts, and the main selection signal.

→ Include what it does, returns, when to use it, when not to, formats and edge cases.

→ With misrouting, fix descriptions before adding infrastructure.

→ Rename overlapping tools. Split multi-purpose ones. Consolidate chains.

→ Check the system prompt for keywords that bias selection.

→ Scope tools by role, and add narrow cross-role tools for frequent needs.

→ Exam: auto, any and forced selection. Today: any and forced return a 400 on Opus 5.5, Sonnet 5.5, Fable 5.1.

→ Today's alternative: auto, strict, a check, or call the step from code.

→ Results: search not list, meaning not UUIDs, verbosity options, paging, helpful truncation.

→ Poka-yoke: change arguments so mistakes are harder.

→ Tool search for bloat, programmatic calling for big results, examples for parameter errors.

→ Evaluate. Only an eval measures selection.

Your notes